

SUSE Observability

Procedimento de implantação

Helm e Fleet • versão 6 • setembro de 2026

Instalar, entender e operar. Três charts, perfis curtos de values e detalhes técnicos separados. Material em português; dashboards e README em inglês.

Use os arquivos Markdown do pacote para copiar comandos sem quebras visuais. Preencha caminhos, contextos, endpoints, nomes de integração e credenciais próprias antes da implantação.

Comece por este roteiro. Consulte docs/details quando precisar aprofundar. Os resultados do laboratório e os testes necessários no cliente estão separados em docs/VALIDATION.md. Este pacote customizado não substitui a documentação oficial do produto.

Implantação no cliente: passo a passo

Este procedimento instala os coletores adicionais, cinco dashboards e 33 monitores. O SUSE Observability e o Agent oficial precisam existir antes. Os exemplos são para um cliente novo; numa atualização, preserve nomes de releases, namespaces, values e tokens proprietários já utilizados.

1. Baixar o pacote e preparar a estação

Instale [Helm](#) e [kubectl](#). Tenha acesso aos kubeconfigs de cada cluster. Não é necessário Python nem gerar arquivos para instalar.

Baixe o ZIP da branch `helm-fleet-v6` no GitHub e extraia. Ou execute:

```
git clone --branch helm-fleet-v6 --single-branch \
  https://github.com/kaioneuhauss/suse-observability-dashboards.git
cd suse-observability-dashboards
mkdir -p local
```

Antes de executar comandos, preencha o alvo. Repita este bloco quando mudar de cluster:

```
export KCFG=/CAMINHO/cluster.yaml
export CTX=CONTEXT0-DO-CLUSTER
export AGENT_NS=suse-observability
export CONTENT_NS=suse-observability
kubectl get nodes --kubeconfig "$KCFG" --context "$CTX"
helm list --all-namespaces --kubeconfig "$KCFG" --kube-context "$CTX"
```

AGENT_NS é o namespace do Agent e de seu Secret. CONTENT_NS é o namespace escolhido para o publicador no central. Eles podem ser diferentes. Não duplique uma release ou coletor já existente. A pasta `local/` é ignorada pelo Git; credenciais continuam em Secrets/cofre.

2. Preparar os pré-requisitos por ambiente

No SUSE Observability central

1. Confirme o servidor instalado conforme os [requisitos SUSE](#).
2. Crie uma integração Kubernetes por cluster e siga as instruções geradas para instalar o Agent oficial. Confirme dados atuais na interface.
3. Prepare o token de publicação descrito na seção 3.
4. Para métricas de VMs, disponibilize o receptor OTLP gRPC com TLS. Se já existe, reutilize. Caso contrário, edite `prerequisites/otlp-ingress-values.yaml`: `hostname`, `ingress class`, `Secret TLS` e `Service real`. Mescle esse fragmento nos values completos do servidor e use a mesma versão instalada. Siga o [procedimento oficial de OTLP](#). Essa alteração pertence à instalação do servidor e pode reconciliar seus Pods.
5. Configure SMTP globalmente se quiser e-mail. A regra de notificação será criada manualmente depois dos monitores.

Em todos os clusters monitorados

O Agent deve estar saudável. Confirme a integração e o Secret no namespace correto:

```
kubectl get pods --kubeconfig "$KCFG" --context "$CTX" -n "$AGENT_NS"
kubectl get configmap suse-observability-agent-cluster-name \
  --kubeconfig "$KCFG" --context "$CTX" -n "$AGENT_NS" \
  -o jsonpath='{.data.STS_CLUSTER_NAME}{"\n"}'
kubectl get secret suse-observability-agent-secrets \
  --kubeconfig "$KCFG" --context "$CTX" -n "$AGENT_NS"
```

Guarde o nome retornado: ele será o `clusterName` do values. Confira a existência da chave `STS_API_KEY` sem exibir seu conteúdo. Ajuste os nomes caso sua instalação use outros objetos. Garanta DNS, TLS e acesso de saída para o SUSE e os registries das imagens.

Nos clusters RKE2 que têm Traefik

No Rancher, abra o cluster provisionado, **Edit Config** → **Edit as YAML**, e mescle `prerequisites/rancher-rke2-traefik.yaml` em `spec.rkeConfig.chartValues.rke2-traefik`. Em RKE2 local/importado, mescle `prerequisites/rke2-traefik-values.yaml` no `spec.valuesContent` do `HelmChartConfig` existente. Use a fonte que já administra essa configuração. [Documentação RKE2](#).

O resultado necessário no `values` do Traefik é:

```
metrics:
  prometheus:
    entryPoint: metrics
    addEntryPointsLabels: true
    addServicesLabels: true
ports:
  metrics:
    port: 9100
    expose:
      default: false
```

Aguarde o rollout. Se existem `NetworkPolicies` isolando o Traefik, permita a coleta interna em 9100 preservando as regras `web/websecure`. Não publique essa porta na Internet. O arquivo [de pré-requisitos](#) explica a política opcional do chart. A mudança no Traefik pode reiniciar Pods: planeje a janela.

Nos nós etcd de RKE2/Harvester

Confira os nós com `node-role.kubernetes.io/etcd=true`. Usando o SSH aprovado do cliente, execute em cada um:

```
curl --fail --max-time 5 http://127.0.0.1:2381/metrics
```

Se retornar métricas `etcd`, habilite `etcd.enabled` no perfil. O laboratório já oferecia esse endpoint local; não foi necessário mudar `etcd-expose-metrics` nem acessar a API 2379. Se o endpoint não existir, valide a versão/configuração antes de habilitar a coleta. Não abra o `etcd` publicamente.

No Harvester

1. Instale/reutilize o Agent seguindo a [documentação da sua versão](#). O add-on v1.8 está documentado como Experimental; avalie o método suportado para o cliente.

2. Mantenha a coleta Traefik desligada quando o Harvester usa NGINX.

3. Com as variáveis apontando para Harvester, copie a CA pública do KubeVirt:

```
mkdir -p local/ca
kubectl get configmap kubevirt-ca \
  --kubeconfig "$KCFG" --context "$CTX" -n harvester-system \
  -o jsonpath='{.data.ca-bundle}' > local/ca/kubevirt-ca.crt
openssl x509 -in local/ca/kubevirt-ca.crt -noout -subject -dates
```

Após conferir que a CA é válida e pertence ao ambiente:

```
set -o pipefail
kubectl create configmap kubevirt-metrics-ca \
  --kubeconfig "$KCFG" --context "$CTX" -n "$AGENT_NS" \
  --from-file=ca-bundle=local/ca/kubevirt-ca.crt --dry-run=client -o yaml \
  | kubectl apply --kubeconfig "$KCFG" --context "$CTX" -f -
```

4. Crie/reutilize um Secret de registry `application-collection` nesse namespace, com permissão para baixar a imagem de `dp.apps.rancher.io`. Use a credencial da organização na tela de registry do Rancher/Harvester.

5. Confirme o Service `kubevirt-prometheus-metrics` e seus endpoints prontos em `harvester-system`. Após rotação da CA, atualize a cópia e incremente `kubevirt.ca.revision` no `values`. Não desabilite TLS para contornar erro de certificado.

3. Preparar o publicador uma única vez

Você pode aproveitar a role padrão `stackstate-k8s-troubleshooter`. A [tabela atual da SUSE](#) inclui as permissões de criar, ler, atualizar e excluir dashboards e monitores. Ela também permite outras operações: se o cliente exigir menor privilégio, use as roles dedicadas da referência técnica.

Peça ao administrador um Service Token aprovado. Se ele ainda não existe, o administrador instala/configura sts pela opção **CLI** da própria instância e executa, substituindo contexto e data:

```
sts service-token create --context CONTEXT0-ADMINISTRATIVO \
--name sre-content-publisher \
--roles stackstate-k8s-troubleshooter \
--expiration AAAA-MM-DD
```

Na interface Rancher, no **cluster central** e namespace `CONTENT_NS`, crie um Secret genérico/Opaque:

Campo	Preencher
Nome	suse-observability-publisher-token
Chave	serviceToken
Valor	Token criado, mantido também no cofre do cliente

O perfil central referencia esse mesmo Secret nos dois Jobs. Não é um acesso especial para o assistente: Helm/Fleet precisa dessa identidade para publicar. O token do Agent é de ingestão e tem outra finalidade. A instalação diária não precisa de sts local; o chart executa os comandos.

O fluxo com role padrão foi conferido na documentação e na renderização. No laboratório, preservamos as identidades restritas existentes; numa instância nova, confira as permissões da versão e o primeiro deploy. Veja [preparo completo e alternativa restrita](#).

4. Editar os values e instalar

Cada perfil é curto e herda os defaults, incluindo requests e limits. Não copie os arquivos do laboratório em `charts/*/config/` para um cliente sem adaptar.

A. Plataforma: repetir em cada cluster

```
mkdir -p local/cluster
cp deploy/cluster/values.yaml local/cluster/values.yaml
```

No Harvester, copie `deploy/harvester/values.yaml` no lugar. Edite:

- `clusterName`: nome exato da integração.
- `remoteWrite.url`: URL da sua instância, normalmente terminada em `/receiver/prometheus/api/v1/write`.
- `etcd.enabled`: true somente após o teste do endpoint local.
- `traefik.enabled`: true somente nos clusters com Traefik preparado.

Com KCFG, CTX e AGENT_NS apontando para o alvo:

```
helm upgrade --install sre-platform-telemetry \
./charts/sre-platform-telemetry \
--kubeconfig "$KCFG" --kube-context "$CTX" -n "$AGENT_NS" \
-f local/cluster/values.yaml --wait --timeout 10m
```

Repita no central, nos downstream e no Harvester. Guarde uma pasta de values por cluster, alterando o caminho -f. Não sobreponha a configuração do cliente ao copiar novamente o exemplo.

B. VMs: somente no Harvester

```
mkdir -p local/virtual-machines
cp deploy/virtual-machines/values.yaml local/virtual-machines/values.yaml
```

Edite `clusterName` para o mesmo nome do Harvester e `otlp.endpoint` para `hostname:443`, sem `https://`. Confira nomes de Secret e CA. Com o contexto Harvester:

```
helm upgrade --install kubevirt-otel-collector \
./charts/sre-kubevirt-telemetry \
--kubeconfig "$KCFG" --kube-context "$CTX" -n "$AGENT_NS" \
-f local/virtual-machines/values.yaml --wait --timeout 10m
```

C. Dashboards e monitores: somente no central

```
mkdir -p local/central
cp deploy/central/values.yaml local/central/values.yaml
```

Edite `observability.url`. Confirme os Secrets; numa instalação existente, mantenha os proprietários originais. Os cinco dashboards e todos os monitores são selecionados por padrão. Para uma coleta parcial, selecione dashboards e slugs do catálogo de monitores correspondentes.

Com o contexto central e `CONTENT_NS` correto:

```
helm upgrade --install suse-observability-content \
./charts/suse-observability-content \
--kubeconfig "$KCFG" --kube-context "$CTX" -n "$CONTENT_NS" \
-f local/central/values.yaml --wait --timeout 15m
kubectl logs job/suse-observability-content-dashboards \
--kubeconfig "$KCFG" --context "$CTX" -n "$CONTENT_NS"
kubectl logs job/suse-observability-content-monitors \
--kubeconfig "$KCFG" --context "$CTX" -n "$CONTENT_NS"
```

5. Aceitar e usar

Confirme os Pods de coleta Ready, os dois Jobs concluídos e amostras recentes de todos os alvos. Para clusters com Traefik, execute também:

```
helm test sre-platform-telemetry \
--kubeconfig "$KCFG" --kube-context "$CTX" -n "$AGENT_NS" \
--logs --timeout 3m
```

Na interface SUSE, abra cada dashboard, teste cluster/namespace/aplicação e compare com o inventário real. Após trocar cluster, redefina filtros dependentes para Everything. Use os pontos de interrogação para entender unidades e limites. Gráficos sem dados exigem conferir a fonte e a aplicabilidade; não equivalem automaticamente a zero.

Crie a notificação por e-mail apontando para os monitores do projeto. Valide abertura e recuperação com um teste controlado. O [checklist de aceite](#) diferencia testes do laboratório de testes ainda necessários no cliente.

6. Atualizar ou usar Fleet

Para atualizar, edite o `values` já salvo e repita o mesmo `helm upgrade --install`. Não é necessário recriar tokens a cada execução.

Para Fleet, use os mesmos charts e leve seus perfis para os arquivos selecionados no `fleet.yaml`. Siga o [procedimento curto de Fleet](#): preparar perfis, ajustar seletores, aplicar coletores e depois conteúdo. Escolha Helm manual ou Fleet por release; não deixe os dois concorrendo.

7. Desinstalar

Primeiro, no contexto central, remova o conteúdo SUSE. Use o values que inclui todos os objetos do projeto que deseja remover, inclusive monitores. As duas seções devem estar habilitadas:

```
helm upgrade suse-observability-content \
  ./charts/suse-observability-content \
  --kubeconfig "$KCFG" --kube-context "$CTX" -n "$CONTENT_NS" \
  -f local/central/values.yaml \
  --set lifecycle.action=delete \
  --set lifecycle.confirmDelete=DELETE_MANAGED_OBJECTS \
  --wait --timeout 15m
```

Confira os logs dos dois Jobs e a ausência dos objetos na interface. Depois:

```
helm uninstall suse-observability-content \
  --kubeconfig "$KCFG" --kube-context "$CTX" -n "$CONTENT_NS"
```

Mude as variáveis para cada cluster e remova a plataforma. No Harvester, remova também VMs:

```
helm uninstall sre-platform-telemetry \
  --kubeconfig "$KCFG" --kube-context "$CTX" -n "$AGENT_NS"
# Somente no Harvester:
helm uninstall kubvirt-otel-collector \
  --kubeconfig "$KCFG" --kube-context "$CTX" -n "$AGENT_NS"
```

Preserve SUSE, Agent, namespaces e credenciais compartilhadas. Remova Secrets/CAs exclusivos e revogue o token do projeto após concluir a exclusão. Os Jobs terminados têm TTL de 24h. O [procedimento de remoção](#) inclui limpeza opcional e a ordem correta quando Fleet é o gerenciador.

Material de apoio

- [Guia simples de estudo](#): o que cada peça faz.
- [Resumo de impacto](#): consumo e planejamento.
- [Detalhes técnicos](#): construção, consultas, roles restritas e diagnóstico.